Polynomial Minimax Optimization
Five Iterative Algorithms

Derivations, parameters, and complete MATLAB code

Research reference note for the ECC polynomial example

Purpose. This report formulates the same polynomial minimax problem under standard gradient descent–ascent (GDA), extra-gradient (EG), optimistic GDA (OGDA), consensus optimization (CO), and the proposed skew-corrected GDA. It explains why each algorithm is used, derives every update, specifies its parameters, and embeds complete MATLAB R2022b source listings. This single LaTeX file compiles directly without external code files, bibliography, or figures.

Contents

1 The polynomial minimax problem **1**

2 Common experiment and all parameters **2**

3 Algorithm derivations **2**
3.1 1. Standard GDA . . . 2
3.2 2. Extra-gradient (EG) . . . 3
3.3 3. Optimistic GDA (OGDA) . . . 3
3.4 4. Consensus optimization (CO) . . . 3
3.5 5. Proposed skew-corrected GDA . . . 4

4 Methods at a glance **5**

5 Numerical comparison **5**

6 Complete MATLAB code for all five algorithms **5**
6.1 `poly_gda_field.m`: shared polynomial field . . . 5
6.2 `method_gda.m`: GDA . . . 6
6.3 `method_extragradient.m`: extra-gradient . . . 6
6.4 `method_ogda.m`: optimistic GDA . . . 6
6.5 `method_consensus.m`: consensus optimization . . . 7
6.6 `method_modified_gda.m`: proposed method . . . 7
6.7 `run_polynomial_minimax_comparison.m`: common driver . . . 8

7 Conclusion **9**

1 The polynomial minimax problem

Consider

$$\min_{x\in\mathbb{R}} \max_{y\in\mathbb{R}} \phi_a(x,y), \qquad \boxed{\phi_a(x,y) = \frac{a}{2}x^2 + xy - \frac{x^4}{4} + \frac{x^6}{6} - \frac{a}{2}y^2 + \frac{y^4}{4} - \frac{y^6}{6}.} \tag{1}$$

The experiment uses $a = 1/20 = 0.05$. Its partial derivatives are

$$\phi_x = ax + y - x^3 + x^5, \qquad \phi_y = x - ay + y^3 - y^5.$$

With $z = [x, y]^\top$, the GDA vector field is

$$\boxed{F(z) = \begin{bmatrix} -\phi_x(z) \\ \phi_y(z) \end{bmatrix} = \begin{bmatrix} -ax - y + x^3 - x^5 \\ x - ay + y^3 - y^5 \end{bmatrix} = \begin{bmatrix} F_1(z) \\ F_2(z) \end{bmatrix}.} \tag{2}$$

The original continuous-time GDA flow is $\dot{z} = F(z)$. The Jacobian used for consensus optimization is

$$\boxed{J_F(z) = \begin{bmatrix} -a + 3x^2 - 5x^4 & -1 \\ 1 & -a + 3y^2 - 5y^4 \end{bmatrix}.} \tag{3}$$

At the origin, $F(0) = 0$, $\phi_{yy}(0) = -a < 0$, and $\phi_{xx}(0) - \phi_{xy}(0)^2/\phi_{yy}(0) = a + 1/a > 0$. Thus, for $a > 0$, the origin is a strict local minimax point. The eigenvalues of $J_F(0)$ are $-a \pm \mathrm{i}$, showing local asymptotic stability of the origin for the original continuous-time GDA flow. This does not rule out periodic orbits away from the origin.

2 Common experiment and all parameters

The same 24 initial points are used for all methods:

$$\boxed{z_0^i = 1.2 \begin{bmatrix} \cos(2\pi(i-1)/24) \\ \sin(2\pi(i-1)/24) \end{bmatrix}, \quad i = 1, \ldots, 24.} \tag{4}$$

Every algorithm has the same step size $h = 0.03$. A run succeeds if

$$\boxed{\|F(z_k)\|_2 \le 10^{-3} \quad \text{for some } 0 \le k \le 10^4.} \tag{5}$$

Here, the stopping test uses the *original* GDA vector field, independently of the modification. Medians are computed over successful runs only.

Table 1: Fixed numerical settings and algorithm-specific gains.

Parameter	Value
Polynomial parameter a	$1/20 = 0.05$
Common step size h	0.03
Number of initial conditions	24
Initial-condition radius	1.2
Maximum updates	10^4
Stopping tolerance	10^{-3}
Extra-gradient look-ahead step	$h = 0.03$
OGDA history weights	2 and -1
Consensus correction gain γ	0.2
Proposed correction gain θ	0.5

3 Algorithm derivations

3.1 1. Standard GDA

Basic idea: The minimizer moves opposite to ϕ_x, while the maximizer moves along ϕ_y. Both use the current iterate. Forward Euler applied to $\dot{z} = F(z)$ gives

$$\boxed{z_{k+1} = z_k + hF(z_k).} \tag{6}$$

Equivalently,

$$x_{k+1} = x_k + h(-ax_k - y_k + x_k^3 - x_k^5),$$
$$y_{k+1} = y_k + h(x_k - ay_k + y_k^3 - y_k^5).$$

Parameters: a, h, z_0, tolerance, and iteration budget. There is no correction gain. One evaluation of F is used per update.

3.2 2. Extra-gradient (EG)

Basic idea: EG evaluates the field at a predicted point to account for interactions that can make ordinary GDA oscillate [1]. First predict

$$\boxed{\widetilde{z}_k = z_k + hF(z_k),} \tag{7}$$

then evaluate the predicted field and update from the *original* iterate:

$$\boxed{z_{k+1} = z_k + hF(\widetilde{z}_k).} \tag{8}$$

In components, with $\widetilde{z}_k = (\widetilde{x}_k, \widetilde{y}_k)$,

$$\widetilde{x}_k = x_k + hF_1(z_k), \qquad \widetilde{y}_k = y_k + hF_2(z_k),$$
$$x_{k+1} = x_k + h(-a\widetilde{x}_k - \widetilde{y}_k + \widetilde{x}_k^3 - \widetilde{x}_k^5),$$
$$y_{k+1} = y_k + h(\widetilde{x}_k - a\widetilde{y}_k + \widetilde{y}_k^3 - \widetilde{y}_k^5).$$

Parameters: a, z_0, predictor step size h, corrector step size h, tolerance, and budget. Both step sizes are equal to 0.03 here. Two field evaluations are required per update.

3.3 3. Optimistic GDA (OGDA)

Basic idea: OGDA uses the gradient directions from the current and previous iterations, forming an optimistic correction [2]:

$$\boxed{z_{k+1} = z_k + h[2F(z_k) - F(z_{k-1})].} \tag{9}$$

Its coordinate form is

$$x_{k+1} = x_k + h[2F_1(z_k) - F_1(z_{k-1})],$$
$$y_{k+1} = y_k + h[2F_2(z_k) - F_2(z_{k-1})].$$

For a fully specified implementation, initialize the stored direction as $F_{-1} = F(z_0)$. The first update then equals standard GDA: $z_1 = z_0 + hF(z_0)$. **Parameters:** a, h, z_0, history initialization, tolerance, and budget. The weights 2 and -1 are fixed, with no additional tunable gain. After initialization, one new field evaluation and one stored vector are used per update.

3.4 4. Consensus optimization (CO)

Basic idea: Consensus optimization introduces a correction designed to reduce the squared norm of the game's gradient vector field [3]. Define the residual function

$$L(z) = \frac{1}{2}\|F(z)\|_2^2 = \frac{1}{2}[F_1(z)^2 + F_2(z)^2]. \tag{10}$$

The chain rule gives

$$\nabla L(z) = J_F(z)^\top F(z). \tag{11}$$

Subtract the consensus correction $\gamma\nabla L$ from the GDA direction and use Euler:

$$\boxed{z_{k+1} = z_k + h[F(z_k) - \gamma J_F(z_k)^\top F(z_k)].} \tag{12}$$

Let $p_k = -a + 3x_k^2 - 5x_k^4$ and $q_k = -a + 3y_k^2 - 5y_k^4$. Then

$$J_F(z_k)^\top F(z_k) = \begin{bmatrix} p_k F_1(z_k) + F_2(z_k) \\ -F_1(z_k) + q_k F_2(z_k) \end{bmatrix},$$

and the coordinate updates become

$$\boxed{\begin{aligned} x_{k+1} &= x_k + h[(1 - \gamma p_k)F_1(z_k) - \gamma F_2(z_k)], \\ y_{k+1} &= y_k + h[\gamma F_1(z_k) + (1 - \gamma q_k)F_2(z_k)]. \end{aligned}} \tag{13}$$

Parameters: a, h, z_0, consensus regularization gain $\gamma = 0.2$, tolerance, and budget. The extra Jacobian-transpose–vector product involves second derivatives of the payoff.

3.5 5. Proposed skew-corrected GDA

Basic idea: The method modifies the current GDA direction by a fixed skew-symmetric transformation. Its continuous-time design uses contraction of infinitesimal areas to rule out periodic orbits under appropriate conditions. Define

$$S = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad A_\theta = I + \theta S = \begin{bmatrix} 1 & -\theta \\ \theta & 1 \end{bmatrix}.$$

Forward Euler for $\dot{z} = A_\theta F(z)$ gives

$$\boxed{z_{k+1} = z_k + h(I + \theta S)F(z_k),} \tag{14}$$

or, equivalently,

$$\boxed{\begin{aligned} x_{k+1} &= x_k + h[F_1(z_k) - \theta F_2(z_k)], \\ y_{k+1} &= y_k + h[\theta F_1(z_k) + F_2(z_k)]. \end{aligned}} \tag{15}$$

Since $\det(I + \theta S) = 1 + \theta^2 > 0$, the modified flow has exactly the same equilibria as the original GDA flow.

For this polynomial,

$$\operatorname{tr} J_{A_\theta F}(z) = -2a - 2\theta + 3(x^2 + y^2) - 5(x^4 + y^4). \tag{16}$$

For all real u, $3u^2 - 5u^4 \le 9/20$. Consequently, for $a = 0.05$ and $\theta = 0.5$,

$$\operatorname{tr} J_{A_\theta F}(z) \le -2(0.05) - 2(0.5) + 2(9/20) = -0.2 < 0.$$

Thus the *continuous-time* modified flow is strictly planar 2-contractive with rate 0.2; the Bendixson criterion excludes nonconstant periodic orbits in the simply connected plane. This is not, by itself, a proof of convergence for the fixed-step Euler iteration.

Parameters: a, h, z_0, correction gain $\theta = 0.5$, tolerance, and budget. The update uses one evaluation of F, with only fixed scalar multiplications and additions afterward; no second derivatives are needed.

4 Methods at a glance

Table 2: Update directions in $z_{k+1} = z_k + hD_k$. The field-evaluation count excludes stopping checks.

Method	Direction D_k	New F evaluations
GDA	$F(z_k)$	1
Extra-gradient	$F(z_k + hF(z_k))$	2
Optimistic GDA	$2F(z_k) - F(z_{k-1})$	1
Consensus	$F(z_k) - \gamma J_F(z_k)^\top F(z_k)$	1†
Proposed	$(I + \theta S)F(z_k)$	1

†Consensus additionally computes a Jacobian-transpose–vector product.

5 Numerical comparison

Table 3: Results from 24 uniformly spaced initial conditions, with $h = 0.03$.

Method	Successful runs	Median iterations
GDA	0/24	N/A
Extra-gradient	0/24	N/A
Optimistic GDA	0/24	N/A
Consensus optimization ($\gamma = 0.2$)	24/24	1679
Proposed method ($\theta = 0.5$)	24/24	476

Here N/A means that no tested run met the stopping criterion in at most 10^4 updates. The exact numerical medians for the successful methods are 1678.5 and 475.5, respectively; the table displays rounded integers. The proposed method needs fewer updates in this experiment, but the observation is not a general discrete-time convergence theorem. The algorithms also differ in their per-update computational costs.

6 Complete MATLAB code for all five algorithms

The seven source files are also provided separately in the `matlab` subfolder, but their complete contents are embedded below so this `.tex` file can be compiled independently. The listings are MATLAB R2022b implementations. Keep the seven `.m` files together in one MATLAB working folder when reproducing the computations. No optional MATLAB toolboxes or random seeds are required.

6.1 `poly_gda_field.m`: shared polynomial field

```
function F = poly_gda_field(z,a)
%POLY_GDA_FIELD GDA vector field for the polynomial minimax example.
%   z = [x;y], a > 0, F = [-phi_x; phi_y].

x = z(1);
y = z(2);

F = [ -a*x - y + x^3 - x^5; ...
       x - a*y + y^3 - y^5 ];
end
```

6.2 method_gda.m: GDA

```
function [z,iterations,success] = method_gda(z0,a,h,maxIter,tol)
%METHOD_GDA Simultaneous gradient descent-ascent (Euler update).

z = z0(:);
iterations = NaN;
success = false;

for k = 0:maxIter
    F = poly_gda_field(z,a);
    if norm(F,2) <= tol
        iterations = k;
        success = true;
        return;
    end
    if k == maxIter
        return;
    end
    z = z + h*F;
    if any(~isfinite(z))
        return;
    end
end
end
```

6.3 method_extragradient.m: extra-gradient

```
function [z,iterations,success] = method_extragradient(z0,a,h,maxIter,tol)
%METHOD_EXTRAGRADIENT One prediction and one correction per step.

z = z0(:);
iterations = NaN;
success = false;

for k = 0:maxIter
    F = poly_gda_field(z,a);
    if norm(F,2) <= tol
        iterations = k;
        success = true;
        return;
    end
    if k == maxIter
        return;
    end
    zHalf = z + h*F;
    FHalf = poly_gda_field(zHalf,a);
    z = z + h*FHalf;  % Update from z, not zHalf.
    if any(~isfinite(z))
        return;
    end
end
end
```

6.4 method_ogda.m: optimistic GDA

```
function [z,iterations,success] = method_ogda(z0,a,h,maxIter,tol)
%METHOD_OGDA Optimistic GDA using the previous GDA direction.
%   The initialization F_{-1} = F_0 makes the first step standard GDA.

z = z0(:);
iterations = NaN;
success = false;
Fprevious = poly_gda_field(z,a);

for k = 0:maxIter
    F = poly_gda_field(z,a);
    if norm(F,2) <= tol
```

```
        iterations = k;
        success = true;
        return;
    end
    if k == maxIter
        return;
    end
    zNext = z + h*(2*F - Fprevious);
    Fprevious = F;
    z = zNext;
    if any(~isfinite(z))
        return;
    end
end
end
```

6.5 `method_consensus.m`: consensus optimization

```
function [z,iterations,success] = method_consensus(z0,a,h,gamma,maxIter,tol)
%METHOD_CONSENSUS Consensus optimization with L = 0.5*norm(F)^2.
%   Correction: J_F(z).' * F(z), with consensus gain gamma > 0.

z = z0(:);
iterations = NaN;
success = false;

for k = 0:maxIter
    F = poly_gda_field(z,a);
    if norm(F,2) <= tol
        iterations = k;
        success = true;
        return;
    end
    if k == maxIter
        return;
    end
    x = z(1);
    y = z(2);
    JF = [ -a+3*x^2-5*x^4, -1; ...
            1, -a+3*y^2-5*y^4 ];
    z = z + h*(F - gamma*(JF.'*F));
    if any(~isfinite(z))
        return;
    end
end
end
```

6.6 `method_modified_gda.m`: proposed method

```
function [z,iterations,success] = method_modified_gda(z0,a,h,theta,maxIter,tol)
%METHOD_MODIFIED_GDA Fixed skew-symmetric correction of the GDA field.
%   Correction: (I+theta*S)*F, S = [0 -1; 1 0].

z = z0(:);
iterations = NaN;
success = false;
S = [0 -1; 1 0];
A = eye(2) + theta*S;

for k = 0:maxIter
    F = poly_gda_field(z,a);
    if norm(F,2) <= tol
        iterations = k;
        success = true;
        return;
    end
    if k == maxIter
        return;
```

```
    end
    z = z + h*(A*F);
    if any(~isfinite(z))
        return;
    end
end
end
```

6.7 run_polynomial_minimax_comparison.m: common driver

```
function results = run_polynomial_minimax_comparison()
%RUN_POLYNOMIAL_MINIMAX_COMPARISON Reproduce the 5-method comparison.
%   MATLAB R2022b; no optional toolboxes; no random initialization.
%   Place this file and all method_*.m/poly_gda_field.m in one folder.

a = 1/20;
h = 0.03;
theta = 0.5;
gamma = 0.2;
maxIter = 10000;
tol = 1e-3;
N = 24;
radius = 1.2;

angles = 2*pi*(0:N-1)/N;
initials = radius*[cos(angles);sin(angles)];
methodNames = {'GDA';'Extra-gradient';'Optimistic GDA'; ...
               'Consensus optimization';'Proposed modified GDA'};
iters = NaN(5,N);

for m = 1:5
    for i = 1:N
        z0 = initials(:,i);
        switch m
            case 1
                [~,n,ok] = method_gda(z0,a,h,maxIter,tol);
            case 2
                [~,n,ok] = method_extragradient(z0,a,h,maxIter,tol);
            case 3
                [~,n,ok] = method_ogda(z0,a,h,maxIter,tol);
            case 4
                [~,n,ok] = method_consensus(z0,a,h,gamma,maxIter,tol);
            case 5
                [~,n,ok] = method_modified_gda(z0,a,h,theta,maxIter,tol);
        end
        if ok
            iters(m,i) = n;
        end
    end
end

successCount = sum(~isnan(iters),2);
medianIterations = NaN(5,1);
for m = 1:5
    successful = iters(m,~isnan(iters(m,:)));
    if ~isempty(successful)
        medianIterations(m) = median(successful);
    end
end

results = table(methodNames,successCount,medianIterations, ...
    'VariableNames',{'Method','SuccessfulRuns','MedianIterations'});

fprintf('a=%.2f, h=%.2f, theta=%.2f, gamma=%.2f, N=%d\n', ...
    a,h,theta,gamma,N);
fprintf('Criterion: ||F(z_k)||_2 <= %.0e; maximum %d updates\n\n', ...
    tol,maxIter);
fprintf('%-25s %-12s %s\n','Method','Successes','Median iterations');
for m = 1:5
    if isnan(medianIterations(m))
        medText = 'N/A';
```

```
    else
        medText = sprintf('%d',round(medianIterations(m)));
    end
    fprintf('%-25s %2d/%-9d %s\n', ...
        methodNames{m},successCount(m),N,medText);
end
end
```

After adding the seven files to the MATLAB path, run the following command:

```
results = run_polynomial_minimax_comparison();
```

The returned MATLAB table retains the unrounded median iteration counts, while the console output rounds them as in Table 3.

7 Conclusion

All five schemes act on the same vector field but make different use of the available gradient information: GDA uses the current direction; EG evaluates a predicted direction; OGDA incorporates the previous direction; consensus optimization penalizes the squared residual; and the proposed scheme introduces a fixed skew correction. The proposed method has a first-order implementation and, for this polynomial, a simple continuous-time negative-divergence certificate. The numerical comparison assesses their Euler-type implementations at one fixed step size; it should not be interpreted as a guarantee for all step sizes or initial conditions.

References

[1] B. Grimmer, H. Lu, P. Worah, and V. Mirrokni, *Limiting Behaviors of Nonconvex-Nonconcave Minimax Optimization via Continuous-Time Systems*, in Proceedings of the 33rd Conference on Algorithmic Learning Theory, PMLR, vol. 167, pp. 465–487, 2022.

[2] C. Daskalakis, A. Ilyas, V. Syrgkanis, and H. Zeng, *Training GANs with Optimism*, International Conference on Learning Representations (ICLR), 2018.

[3] L. Mescheder, S. Nowozin, and A. Geiger, *The Numerics of GANs*, Advances in Neural Information Processing Systems, vol. 30, 2017.